

IEEE x Knight Hacks Embedded Workshop!

Session 1: GPIO, ADC, and PWM Basics



[IEEE Discord Link](https://discord.gg/hAGRkgEBZQ)



[Github Link](#)

What's your level?

PREREQUISITE: You ***MUST*** have a laptop that can run the Arduino IDE.

Level 1: I don't know how to code and I don't know Embedded. :(

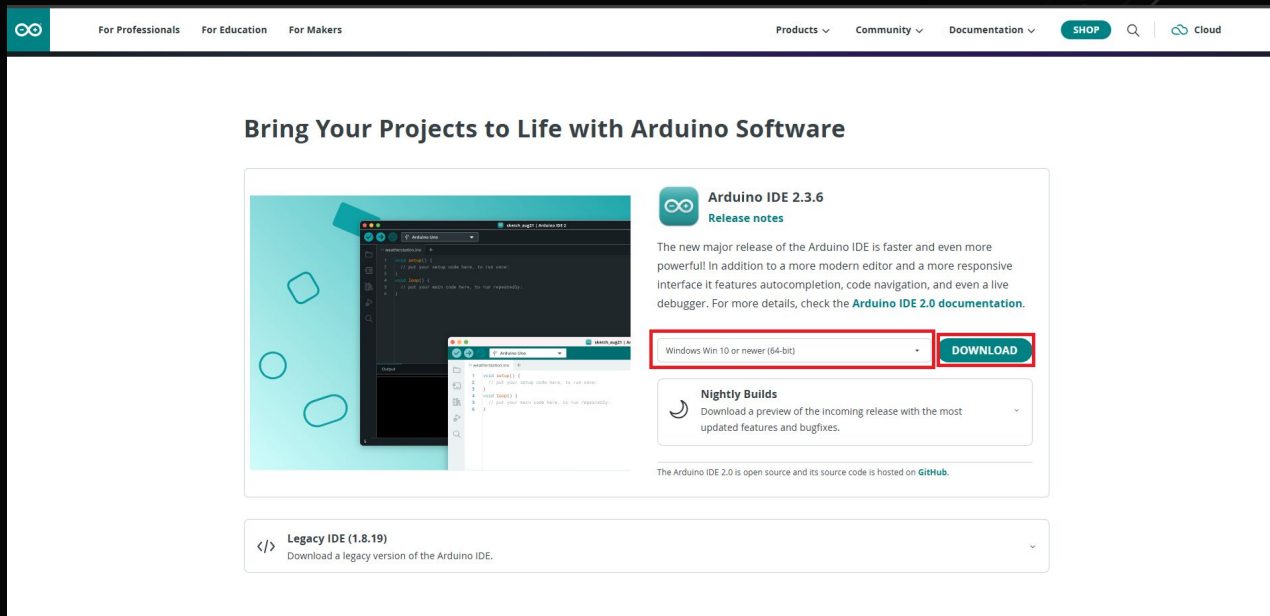
Level 2: I've never programmed a microcontroller before, but I know how to code!

Level 3: I know how to flash code to a board! I can write and upload firmware, use serial communication, understand GPIOs, and read datasheets without crying. I also know how to breadboard!

Summary of Topics Covered!

- How to install the Arduino IDE and configure it for use with ESP32 MCU
- How to connect the components on the breadboard
- Basics of embedded programming structure
- Basics of GPIO pins and usage
- Basics of ADC input and potentiometers
- How to put it together for LED control

How to Install and Configure the Arduino IDE



Bring Your Projects to Life with Arduino Software

Arduino IDE 2.3.6
Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit) **DOWNLOAD**

Nightly Builds
Download a preview of the incoming release with the most updated features and bugfixes.

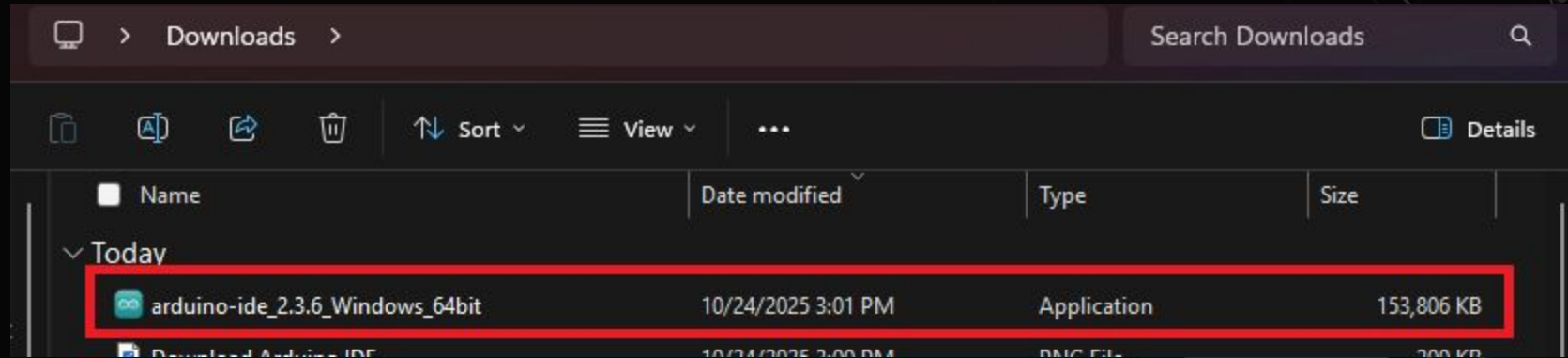
The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Legacy IDE (1.8.19)
Download a legacy version of the Arduino IDE.

Select your OS and click “DOWNLOAD”

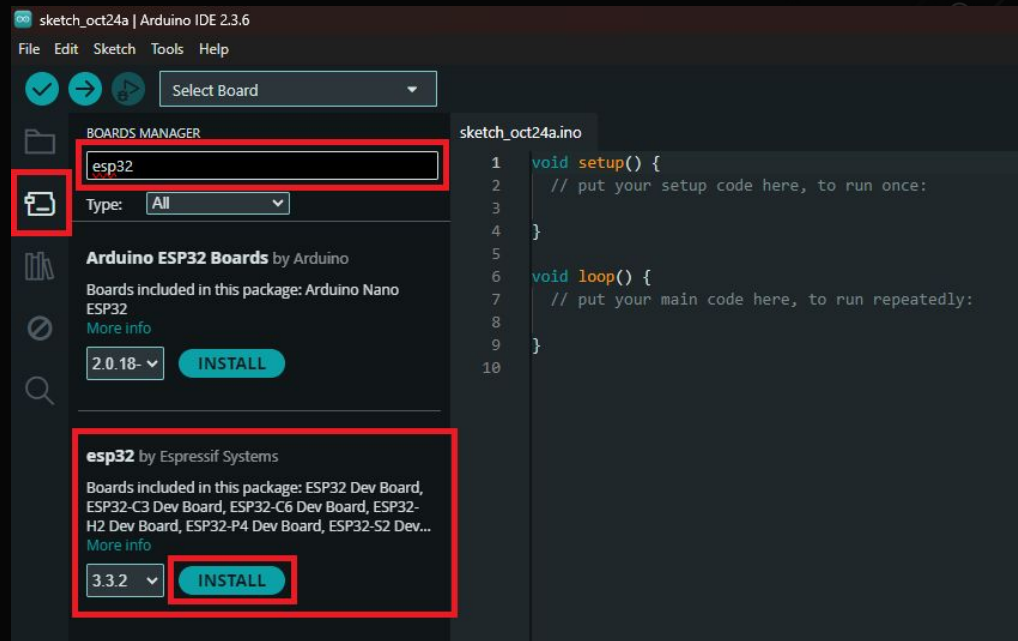
Link: <https://www.arduino.cc/en/software/>

How to Install and Configure the Arduino IDE continued...



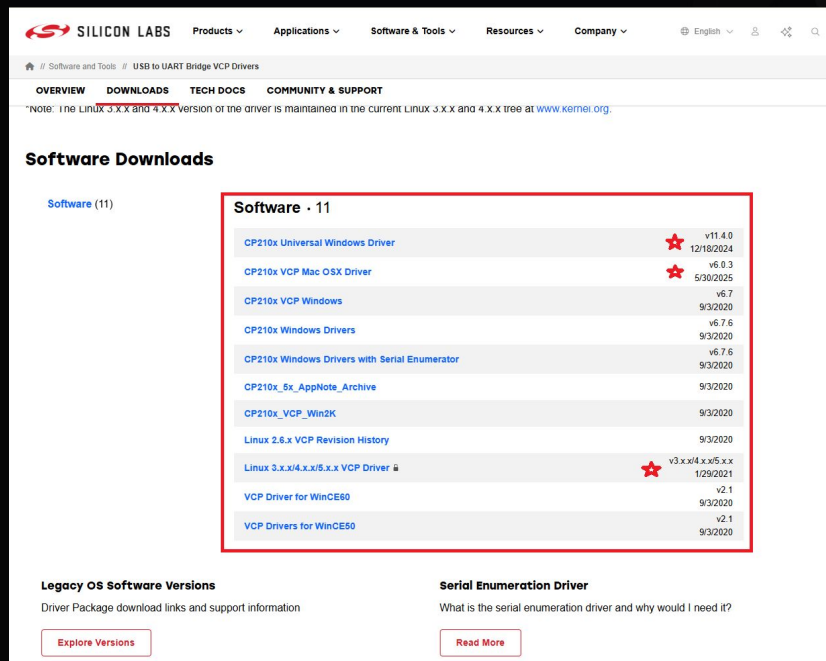
Next, run the installer!

How to Install and Configure the Arduino IDE continued...



Open the IDE > Select the “Boards” tab > search “ESP32” > Install the “esp32” pack by Espressif Systems

How to Install and Configure the Arduino IDE continued...



SILICON LABS Products Applications Software & Tools Resources Company

Software and Tools # USB to UART Bridge VCP Drivers

OVERVIEW DOWNLOADS TECH DOCS COMMUNITY & SUPPORT

*Note: The Linux 3.x.x/4.x.x/5.x.x version of the driver is maintained in the current Linux 3.x.x and 4.x.x tree at www.kernel.org.

Software Downloads

Software (11)

Software · 11

CP210x Universal Windows Driver	★	v11.4.0 12/18/2024
CP210x VCP Mac OSX Driver	★	v6.0.3 5/30/2025
CP210x VCP Windows		v6.7 9/3/2020
CP210x Windows Drivers		v6.7.6 9/3/2020
CP210x Windows Drivers with Serial Enumerator		v6.7.6 9/3/2020
CP210x_5x_AppNote_Archive		9/3/2020
CP210x_VCP_Win2K		9/3/2020
Linux 2.6.x VCP Revision History		9/3/2020
Linux 3.x.x/4.x.x/5.x.x VCP Driver	★	v3.x.x/4.x.x/5.x.x 1/29/2021
VCP Driver for WinCE60		v2.1 9/3/2020
VCP Drivers for WinCE50		v2.1 9/3/2020

Legacy OS Software Versions

Driver Package download links and support information

[Explore Versions](#)

Serial Enumeration Driver

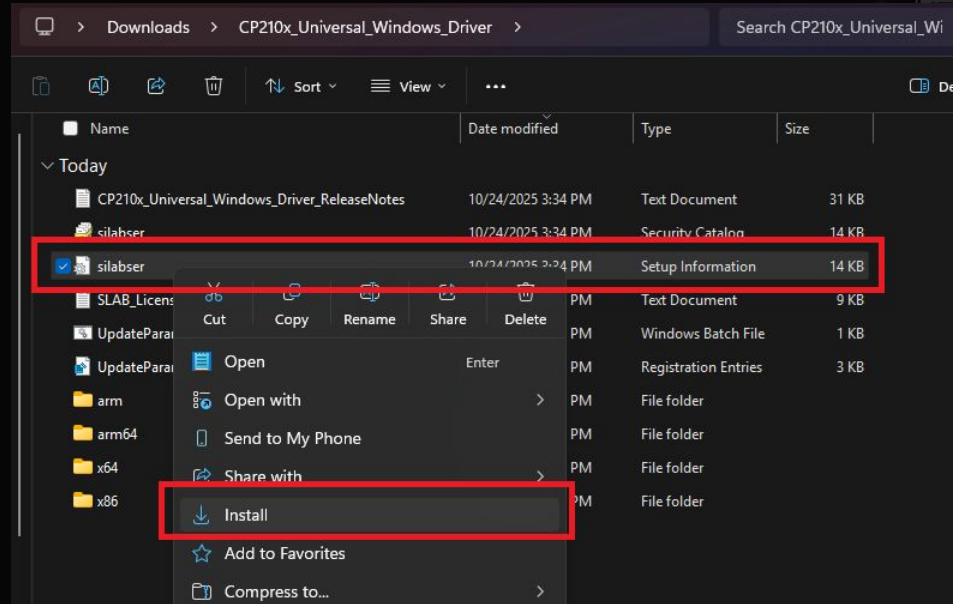
What is the serial enumeration driver and why would I need it?

[Read More](#)

Then download the appropriate driver based on your OS

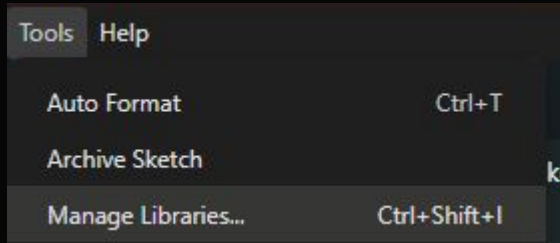
Link: <https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads>

How to Install and Configure the Arduino IDE continued...

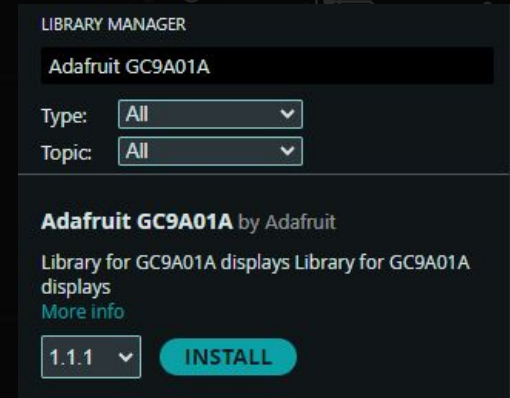


Extract the folder > Run the “silabser” .inf/setup information file installer (right-click and select “install”)

How to Install and Configure the Arduino IDE continued...

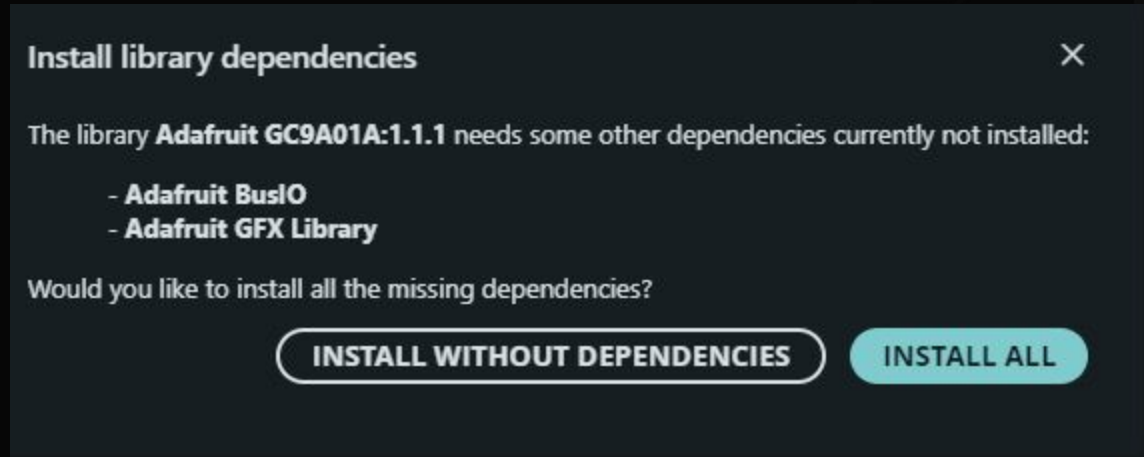


Once you load the IDE,
go to tools and go to
manage libraries.



You will see a library
manager pop up, in
the search bar, type in
"Adafruit GC9A01A"

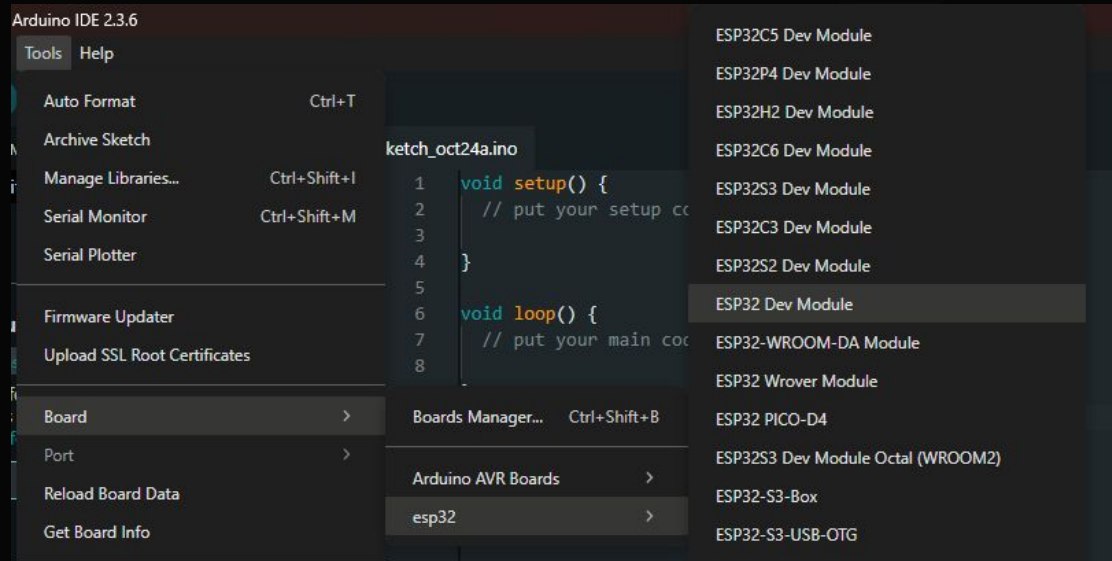
How to Install and Configure the Arduino IDE continued...



It will ask you to install 2 additional libraries, press **INSTALL ALL**.

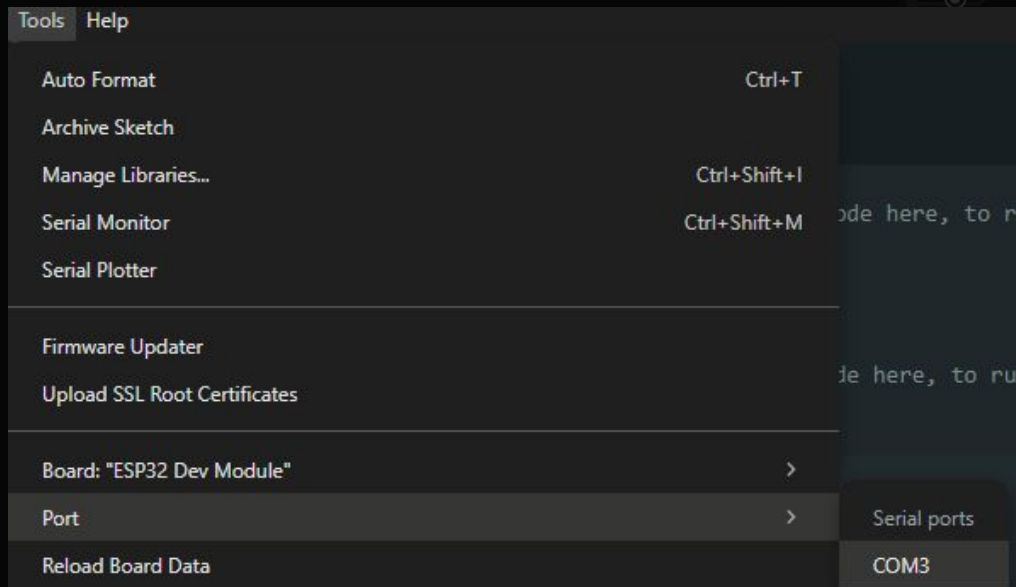
The Display in Session 2 will not work without it. :(

How to Install and Configure the Arduino IDE continued...



Go back to tools, then go the the board section, then hover over ESP32, then select “ESP32 Dev Module”

How to Install and Configure the Arduino IDE continued...

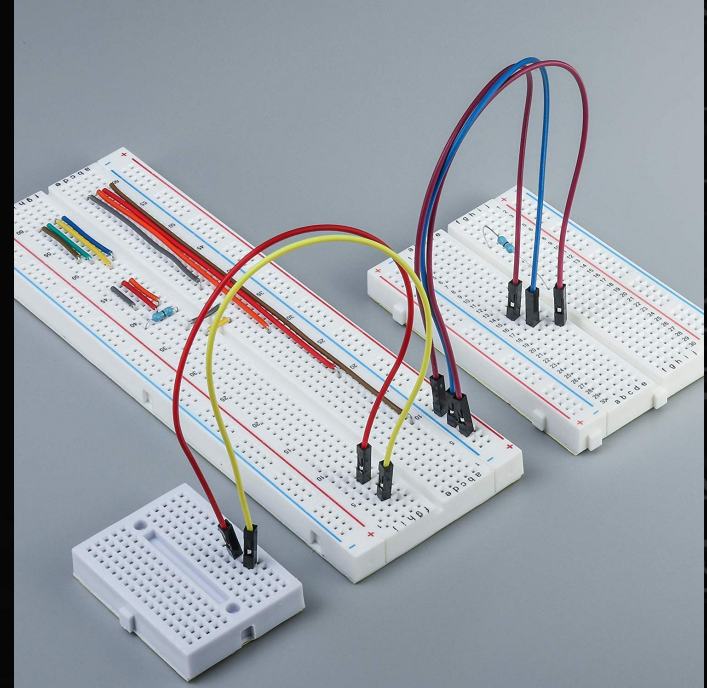


Then, once you have selected the correct board, you will see you finally have the option to select a Port. Select the COM3 Port.

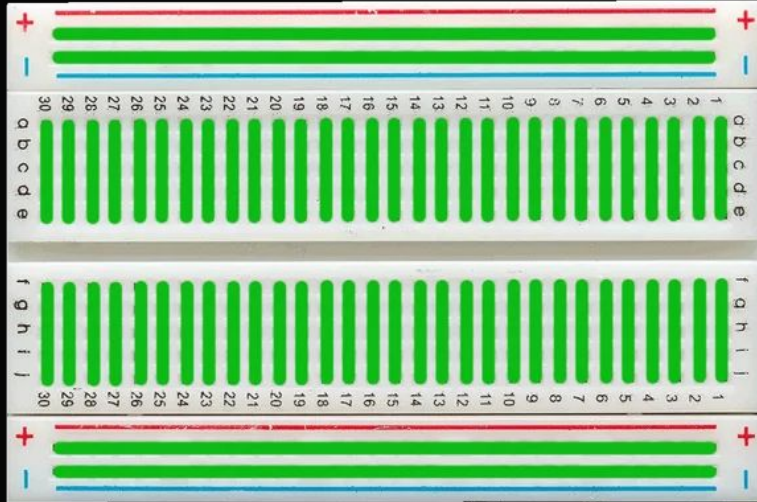
And then you're done! :)

How to Connect the Components on the Breadboard!

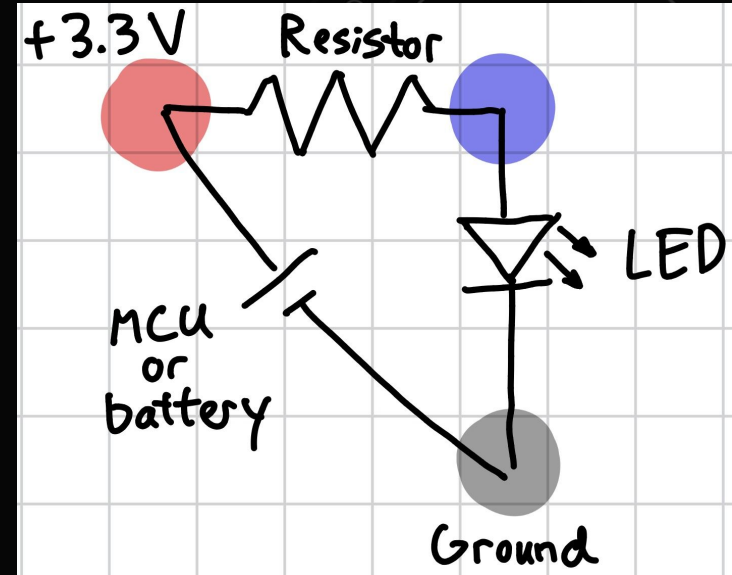
- So.. what is a solderless breadboard?
 - It's a tool that allows us to connect our MCU (microcontroller) and components easily.
- But how do we connect things?
 - Certain rows/columns of the breadboard are connected, we plug wires into the same rows/columns to connect them!



How to Use the Breadboard Continued..

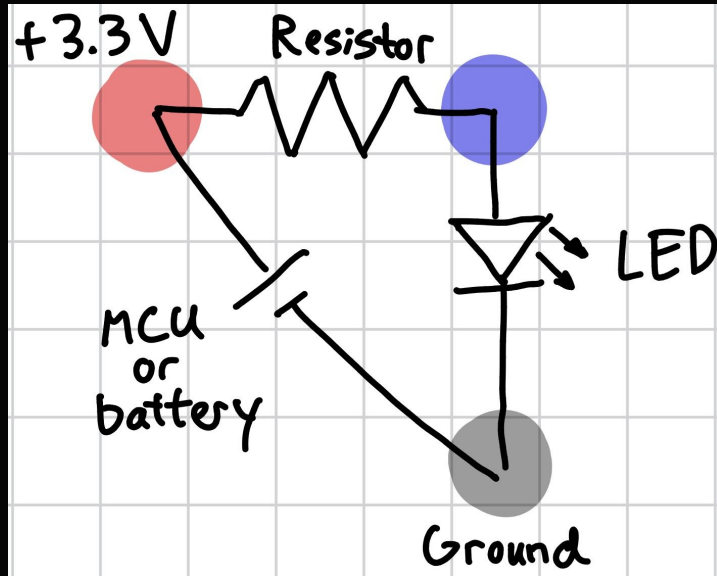


The green lines show the connections on the breadboard

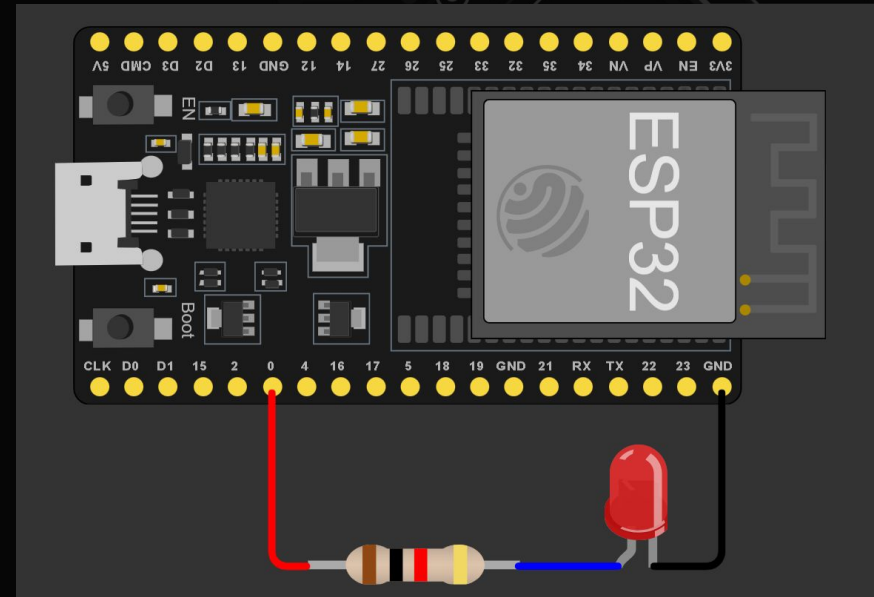


Circuit schematics show the components in their symbol form and their connections

How to Use the Breadboard Continued..

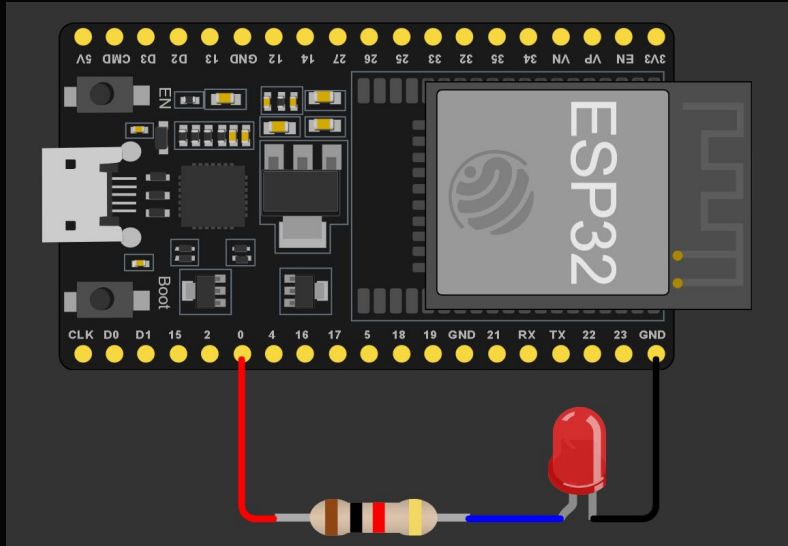


Circuit schematics show the components in their symbol form and their connections

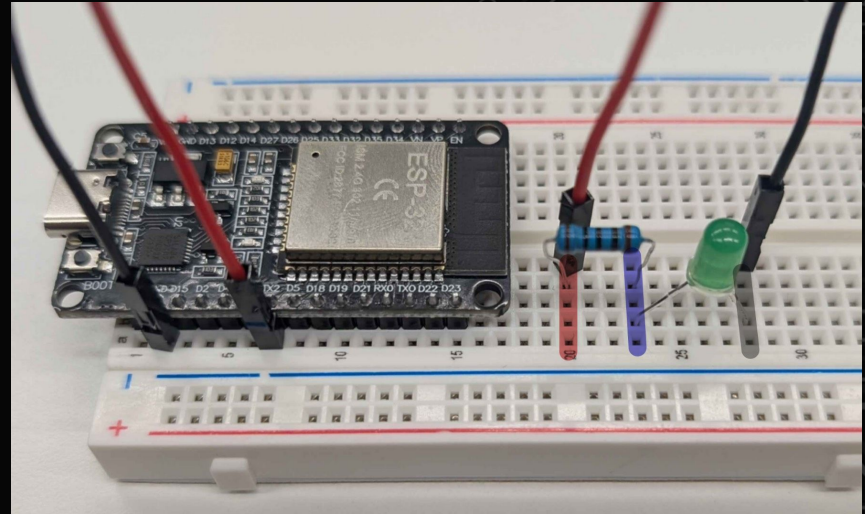


We can realize these connections using our actual components by soldering or custom PCBs

How to Use the Breadboard Continued..



We can realize these connections using our actual components by soldering or custom PCBs

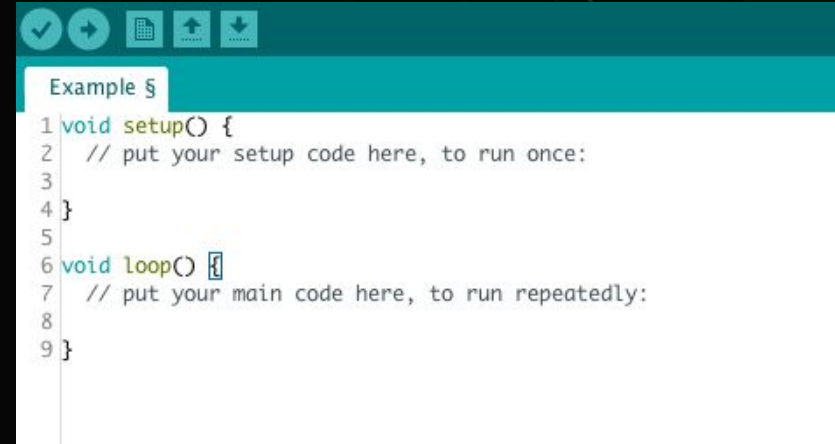


The breadboard can be used instead to make the connections easy!
*Note the highlighted lines

Structure of an Embedded Program

There are two main phases of an Embedded Program:

- **setup() - Initialization (runs once at startup!)**
 - Configure pin directions (input/output)
 - Initialize peripherals: UART, SPI, I²C, ADC, PWM, etc..
 - Start timers, interrupts, and libraries
 - Prepare sensors or communication modules
- **loop() – Execution (runs continuously)**
 - Read sensors and update variables
 - Run control logic (if/else, PID, FSMs, etc.)
 - Send or display data (serial, LCD, LEDs)
 - React to events and adjust outputs in real time



```

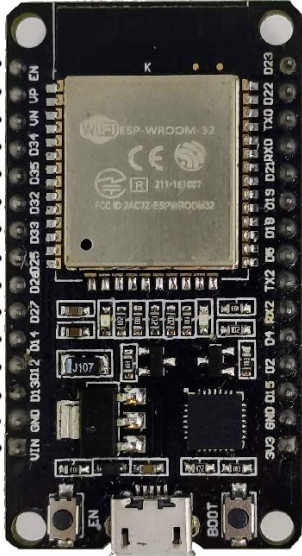
Example §
1 void setup() {
2   // put your setup code here, to run once:
3
4 }
5
6 void loop() {
7   // put your main code here, to run repeatedly:
8
9 }
  
```

Basics of GPIO Pins and Usage

ESP32 DEV KIT V1

PINOUT

	EN	5
Input only	RTC GPIO0	SensVP
Input only	RTC GPIO3	SensVN
Input only	RTC GPIO4	ADC1_6
Input only	RTC GPIO5	ADC1_7
RTC GPIO9	Xtal32P	Touch9
RTC GPIO8	Xtal32N	Touch8
DAC 1	RTC GPIO6	ADC2_8
DAC 2	RTC GPIO7	ADC2_9
RTC GPIO17	Touch7	ADC2_7
RTC GPIO16	Touch6	HSPI_CLK
RTC GPIO15	Touch5	HSPI_Q
RTC GPIO14	Touch4	HSPI_ID
	GND	
	VIN	



36	GPIO23	V_SPI_D	MOSI
39	GPIO22	V_SPI_WP	SCL
41	GPIO1	TXD 0	CLK3
40	GPIO3	RXD 0	CLK2
42	GPIO21	VSPI_HD	SDA
38	GPIO19	V_SPI_Q	MISO
35	GPIO18	V_SPI_CLK	SCK
34	GPIO5	V_SPI_CS0	SS
27	GPIO17	TXD 2	
25	GPIO16	RXD 2	
24	GPIO4	ADC2_0	HSPI_HD
22	GPIO2	ADC2_2	HSPI_WP0
21	GPIO15	ADC2_3	HSPI_CS0
	GND		
	3.3v		

Touch0	RTC GPIO10
Touch2	RTC GPIO12
Touch3	RTC GPIO13



www.mischianti.org



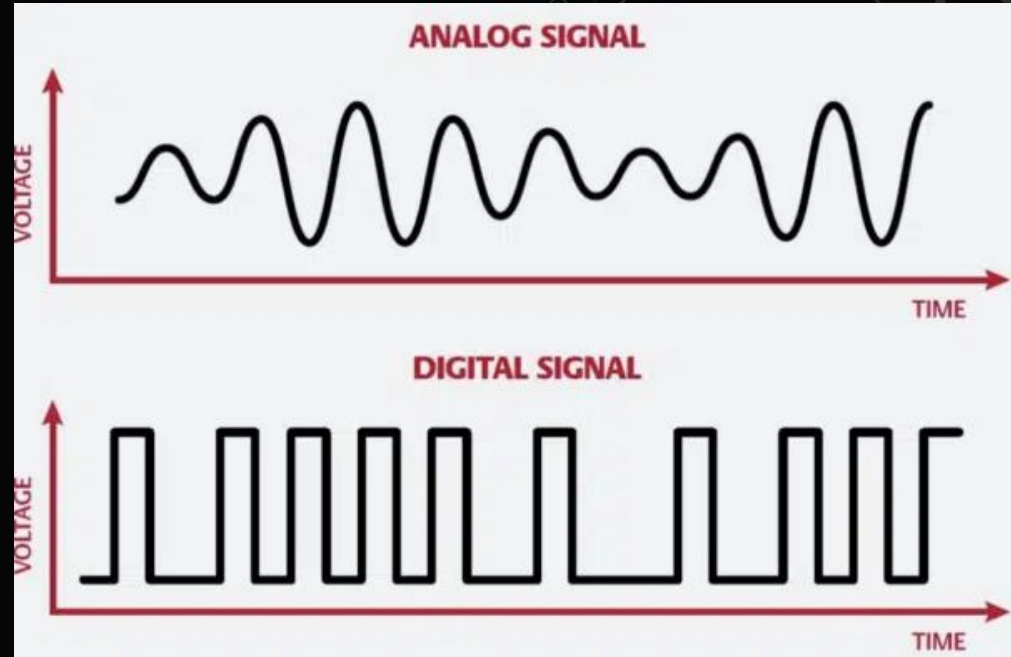
GPIO (General Purpose Input/Output) - the foundation for using an MCU

Basics of GPIO Pins and Usage cont.

There are two kinds of signals:

Analog - continuous voltage level to digital value (0-3.3V maps to 2^n values)

Digital - discrete voltage levels (0 and 3.3V maps to logic 0 and logic 1)



Basics of GPIO Pins and Usage cont.

INPUT



Example:
Press and release pushbutton

- `pinMode(pin, INPUT);`
- `pinMode(pin, INPUT_PULLUP);`
- `digitalRead(pin)`
- Returns 0 or 1

DIGITAL



OUTPUT

- `pinMode(pin, OUTPUT);`
- `digitalWrite(pin, 0 or 1)`
- Write 0 or 1

Example:
Turn the LED on and OFF

INPUT



Example:
Rotate the rod of the potentiometer

- `analogRead(pin)`
- Returns 0 - 1023



ANALOG



OUTPUT

- `analogWrite(pin, 0-255)`
- Write 0 - 255



Example:
Fade the LED's brightness on and off

More Core Functions in Embedded Programming

pinMode(pin, mode) - Configures a GPIO pin as INPUT, OUTPUT, INPUT_PULLUP

- Must be in setup() before use

digitalRead(pin) / digitalWrite (pin, value) - Handles digital I/O (Input/Output)

- Sets or reads pins values as logic 1 (high) or 0 (low), 3.3V or 0V

analogRead(pin) / analogWrite(pin,value) - Reads and writes voltage levels to the pins (3.3-0V)

- Sets or reads pins voltages between 3.3V - 0V

ledcWrite(channel, value) - Generates a PWM output

- Can vary LED brightness, motor position, etc.
- Values between 0-255 (8 bit) | frequency/resolution configurable in ledcSetup()

More Core Functions in Embedded Programming cont.

millis() - Returns the number of milliseconds since startup.

- Used for non-blocking timing (that means means your program can keep doing other things while it's waiting for time to pass!) and event scheduling instead of delay().
- Lets you run multiple tasks in parallel (ex: blinking + sensor reading).

map(x, in_min, in_max, out_min, out_max) - Converts a value from one range to another.

- An example would be mapping potentiometer input (0 - 4095) to led brightness (0 - 255). You'll see this in another example!

Serial.begin(baud) / Serial.print() - Enables serial communication.

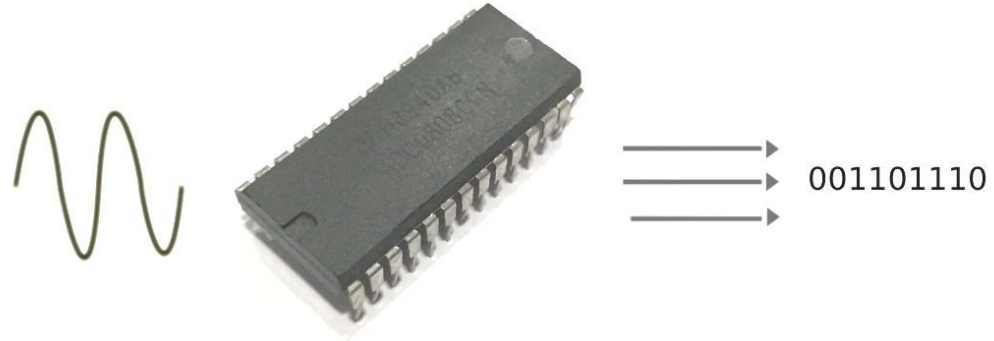
- Baud just means the speed of the communication.
- Both of these functions are used for debugging, printing sensor data, and monitoring variables.
- You use it to print stuff to the serial monitor! (You'll see more of this in day 3.)
- baud starts the conversation, it sets up the UART hardware and decides how fast it talks, like `Serial.begin(115200);`
- print actually sends the messages, like
`Serial.print("Temperature: ");`
`Serial.println(sensorValue);`

Basics of ADC Input and Potentiometers

What is an ADC?

- Its an Analog to Digital Converter!
- It reads a voltage level (0-3.3V) and maps it to (0-4095 bits)
- It's used to read sensors, knobs, and Much more!

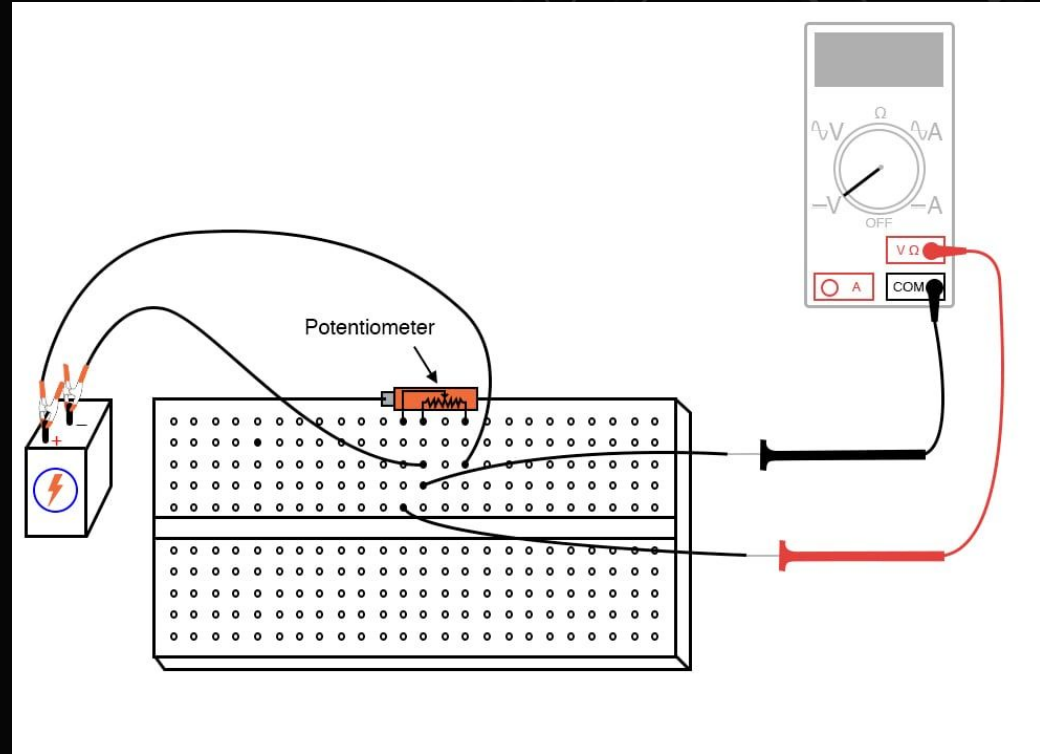
Analog to Digital Converters



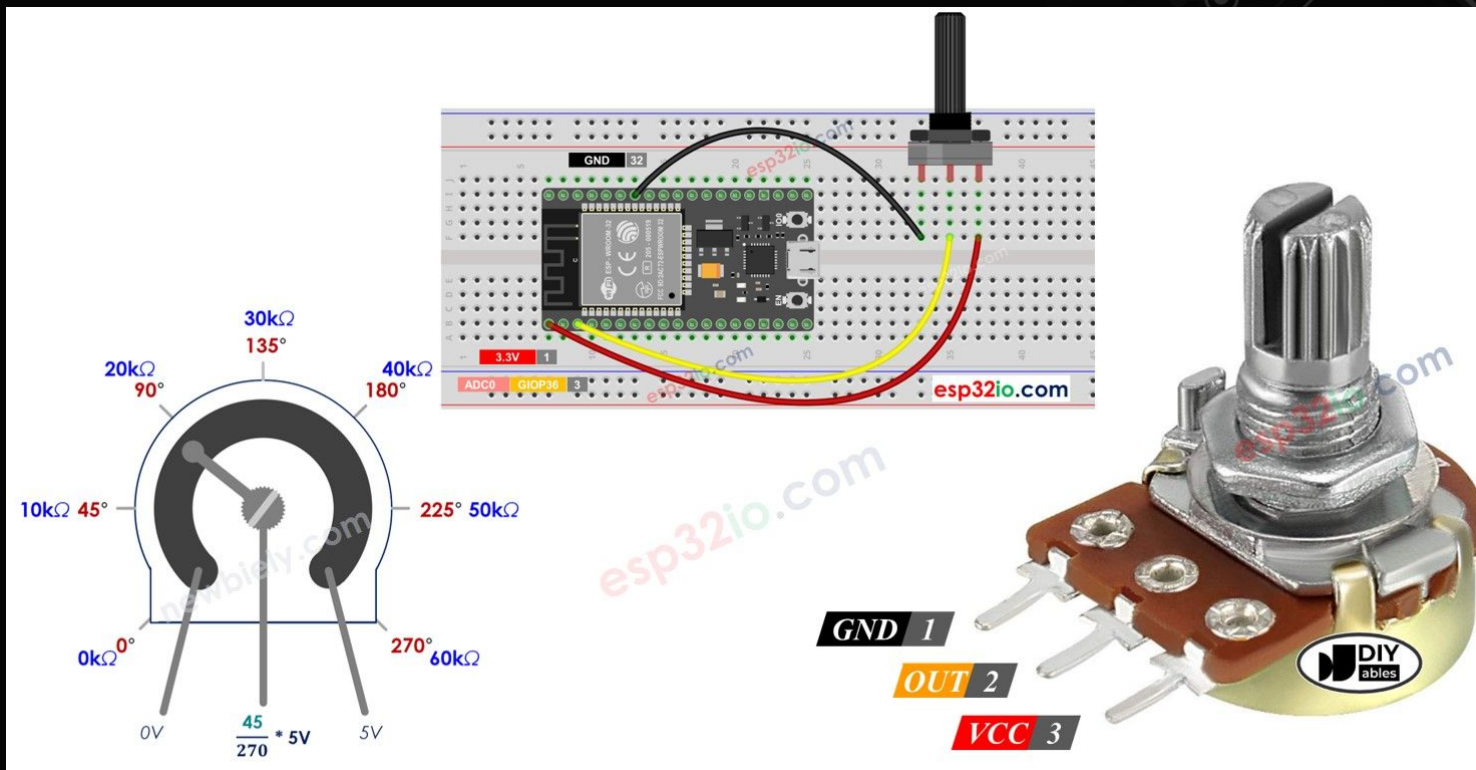
Basics of ADC Input and Potentiometers

What is a Potentiometer?

- It acts like a voltage divider! (it splits the voltage based on how much you turn the knob)
- The middle pin gives variable voltage depending on knob position
- You connect the ends to 3.3V and GND, and the middle pin goes to the ADC input.

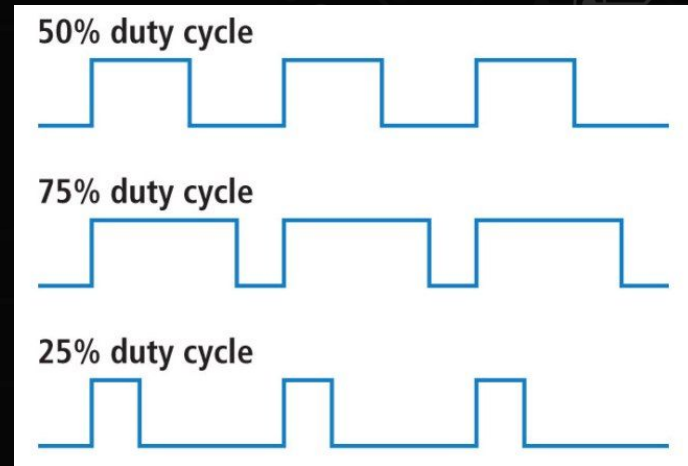


Basics of ADC Input and Potentiometers Continued...



Pulse Width Modulation (PWM)

- How do we dim an LED? An easy method is Pulse Width Modulation (PWM)
- PWM is a type of modulation in which the width of a pulse that occurs at a constant interval is modified to become larger to thinner.
- The interval between pulses remains the same, but pulse width changes
- The **duty cycle** of a PWM signal refers to how long a pulse stays on (voltage high) in a single pulse cycle
- For an LED, the larger the duty cycle, the brighter the LED.



How to Put it All Together for LED Control

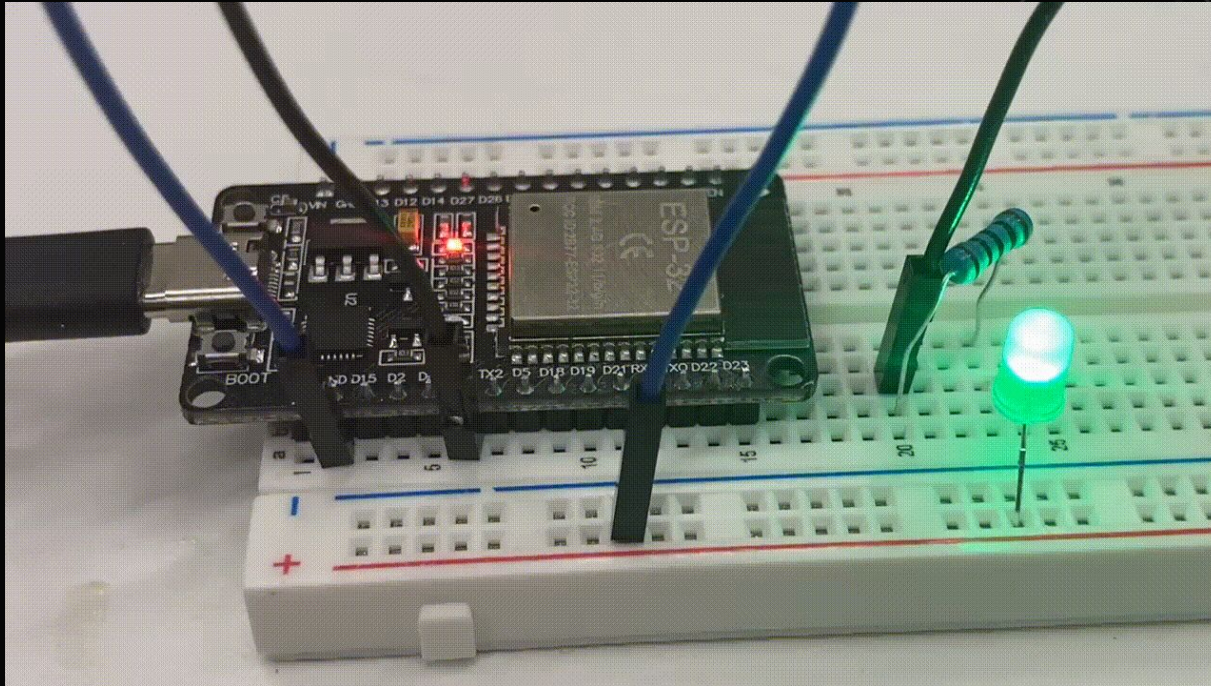
1. (Optional) Define an output pin as a macro
2. Configure the pin to be output in the setup function
3. Toggle the pin with a delay
4. (Optional) Add in brightness controlled PWM LEDs or other functionality

Let's look at an example!

Example: A Blinking LED!

```
1  #define BLINK_LED 16 // The GPIO Pin that you are connected to, like GPIO 16.
2
3  void setup() {
4      pinMode(BLINK_LED, OUTPUT); // set the LED pin as output, so we can turn it on and off
5  }
6
7  // okay, so this will use some stuff we didn't talk about yet.
8  // ledState keeps track of whether the LED is currently ON or OFF
9  // lastBlink stores the last time (in ms) the LED changed state
10
11  bool ledState = false; // basically "is the LED on?"
12  unsigned long lastBlink = 0; // how long it's been since it blinked
13
14  void loop() {
15      if (millis() - lastBlink >= 500) { // check if 500ms passed since last toggle
16          ledState = !ledState; // flip LED state (so ON becomes OFF, OFF becomes ON)
17          digitalWrite(BLINK_LED, ledState); // actually apply the new LED state to the pin
18          lastBlink = millis(); // record the current time (so the timer resets)
19      }
20  }
```


Try it yourself!



Challenge!

Try making one LED blink, and another LED with adjustable brightness using PWM!

(We can't show you what it looks like, make it yourself!)

```
#include <Arduino.h>

#define BLINK_LED 16 // GPIO16 (RXD 2)
#define PWM_LED 17 // GPIO17 (TXD 2)
#define POT_PIN 4 // GPIO_4

// you can choose whatever pins you want, but we recommend these!

bool ledState = false; // tracks LED on/off state
unsigned long lastBlink = 0; // for blink timing

void setup() {
  Serial.begin(115200); // start serial monitor for debugging
  pinMode(BLINK_LED, OUTPUT); // setup blink LED as output

  // attach PWM to PWM_LED at 5kHz with 8-bit resolution
}

void loop() {
  // read potentiometer value (0-4095)
  // map it to 8-bit brightness (0-255)
  // update PWM LED brightness

  // check if time passed using millis()
  // if yes, toggle ledState and update BLINK_LED
}
```

QR Codes!!!!



[Feedback Link](#)



[IEEE Discord Link](#)